Universidad de Carabobo
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación

Práctica de Laboratorio Número 4

Profesor:
José Canache
Arquitectura del Computador

Autor:
Ronald Montilla
C.I. 32.011.453

5 de Octubre de 2026

Índice

1. E/S por interrupciones o sondeo **2**

2. Ciclo de Interrupción por Hardware **3**

3. Interrupciones y excepciones en MIPS32 **4**
3.1. Registros para gestionar interrupciones en MIPS32 4
3.2. Generación de excepciones 6
3.3. Tratamiento de excepciones 6

4. Análisis y discusión de resultados **7**

5. Bibliografía **8**

1. E/S por interrupciones o sondeo

Las E/S por interrupciones o sondeo son dos conceptos sumamente importantes, estos permiten la entrada/salida de dispositivos externos de maneras muy distintas, con sus ventajas y desventajas.

Sondeo o polling:

En primer lugar esta la E/S por sondeo(*polling*). Es un método que consiste en preguntar constantemente a un dispositivo periférico si ya está disponible para la acción de E/S. Es un método muy sencillo, pues solo se mantiene preguntando al dispositivo externo si ya está listo y hasta que este no responda, el procesador no avanza a instrucciones diferentes. Las principales ventajas de este método son su sencillez y funcionalidad, es decir, funciona y hace perfectamente su trabajo, pero tiene dos grandes desventajas, en primer lugar consume muchos ciclos de reloj innecesarios al estar constantemente interactuando con el dispositivo externo en busca de la respuesta que busca, la segunda gran desventaja es su negativa eficiencia ante un procesador que debe atender muchos dispositivos E/S, esto se debe a que si el procesador esta en estado de sondeo no podrá atender a los demás periféricos. Además, la E/S por sondeo puede provocar la pérdida de eventos ante dispositivos que generan la señal y luego cambian el estado justo en el momento en el que el procesador no esta preguntando.

Interrupción:

Por otro lado esta la E/S por interrupciones, este método es lo contrario al *polling*, el procesador no espera al dispositivo, en cambio, el procesador ejecuta instrucciones hasta que se produzca una *interrupción*. Una interrupción es una señal eléctrica que envía un dispositivo externo al procesador como una petición para una acción de entrada salida. Es decir, el procesador no entra en un bucle esperando a que el dispositivo envíe la señal, mejor emplea esos ciclos de CPU en realizar instrucciones. Al momento de producirse una interrupción la CPU primero termina la instrucción que estaba realizando, luego guarda su estado actual y el contador del programa en la pila, y posterior a esto, guarda el contexto(registros a emplear) y ejecuta las instrucciones E/S.

Pero surge una pregunta muy importante, ¿Por qué es necesario guardar el contexto (registros) al entrar en una rutina de servicio? Debemos saber que mientras el procesador se encuentra ejecutando un programa emplea datos y registros para las instrucciones que efectúa y las operaciones que realiza a lo largo de todo el programa, pero cuando de produce una interrupción el procesador cambia de lugar hacia una rutina de servicio. En esta rutina de servicio el procesador ejecuta las instrucciones y operaciones de E/S, empleando durante este tiempo datos y registros que posiblemente estaba empleando el programa que fue interrumpido. Entonces, el procesador debe guardar ese contexto por esa razón, porque si no lo hace puede perder esos datos y registros importantes para el programa que fue interrumpido.

La interrupciones son muy importantes en la actualidad, debido a la tendencia multitarea de los procesadores actuales. Gracias a las interrupciones el procesador puede ejecutar instrucciones eficientemente al aprovechar ciclos de reloj que el sondeo, por otro lado, desperdicia. Algunas de las principales ventajas de E/S por interrupción es la eficiencia de ciclos de CPU y mayor seguridad ante perdida de eventos, pero como desventajas podemos encontrar su difícil y complejo hardware para atender estas interrupciones, guardar

contexto y recuperarlo.

2. Ciclo de Interrupción por Hardware

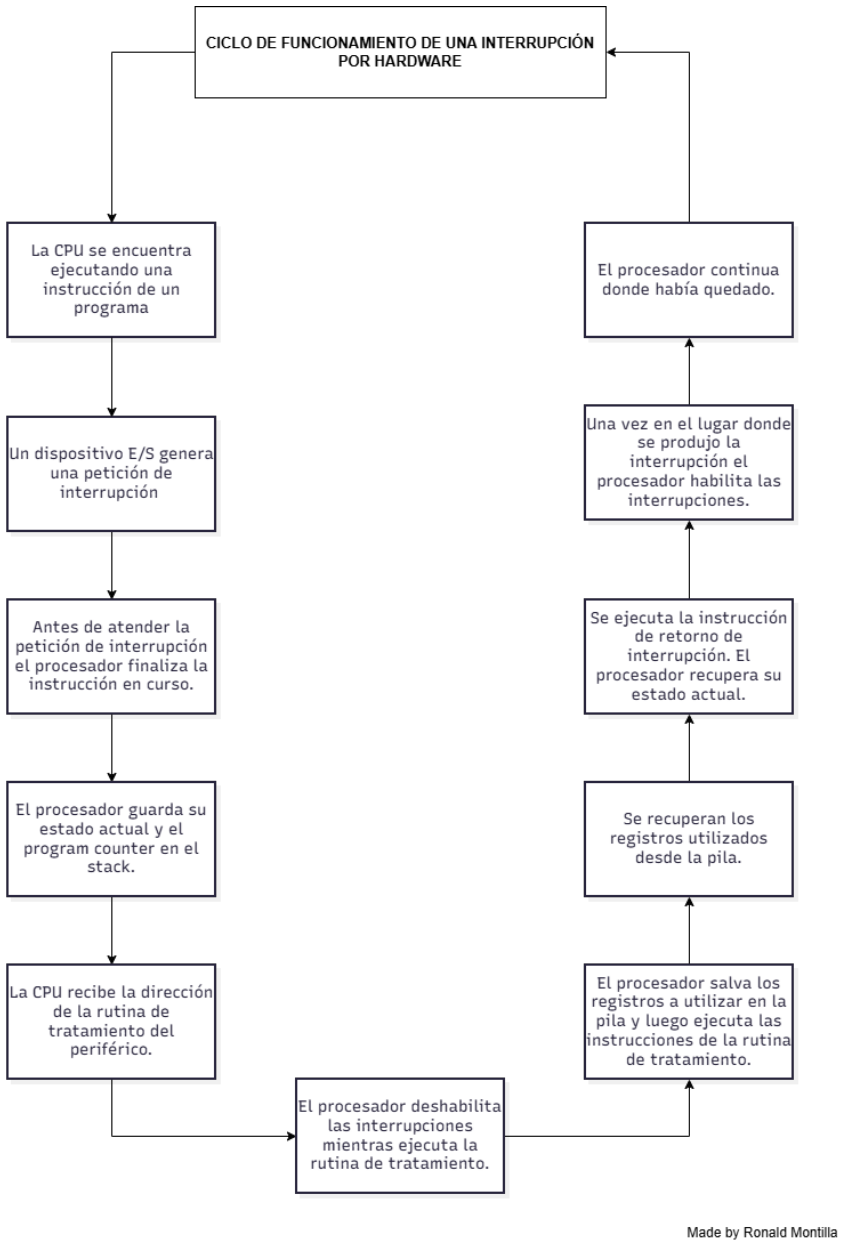


Figura 1: Funcionamiento del ciclo de una interrupción por hardware, desde que se produce hasta que se atiende.

3. Interrupciones y excepciones en MIPS32

3.1. Registros para gestionar interrupciones en MIPS32

En MIPS32 contamos con los registros del Coprocesador(CP0) para gestionar las interrupciones. Existen 32 registros especiales del Coprocesador, sin embargo, son tres los registros que nos permitirán gestionar las interrupciones.

- Registro **status(Numero 12)**: Este registro nos permite activar o desactivar las interrupciones a través de la manipulación de sus bits. En este registro de 32 podemos encontrar lo siguiente:

  Interruptor Enable(IE) - Bit 0: Si se establece en 1, las interrupciones se activan. Pero en 0 permanecen activadas.

  Exception Level(EXL), bit 1: El procesador lo pone a 1 automáticamente cuando ocurre una excepción o interrupción. Mientras esté en 1, fuerza modo kernel y bloquea nuevas interrupciones para evitar reentradas descontroladas.

  (Error Level(ERL), bit 2): Utilizado para errores críticos de hardware.

  Interrupt mask(IM), bits 15:8, Máscara individual de interrupciones. IM[1:0], interrupciones por software. IM[7:2], interrupciones por hardware externas.

- Registro **cause(numero 13)**: Este registro nos permite saber las causas que originaron la interrupción. Basando en lo siguiente:

  (Exception Code, bits 6:2): Indica la causa básica. Para interrupciones por hardware o software, su valor es 0.

  (Interrupt Pending, bits 15:8): Muestra qué líneas de interrupción están pidiendo atención en este instante (IP[1:0] software, IP[7:2] hardware). Una interrupción se dispara si (IP & IM) != 0 y tanto IE == 1 como EXL == 0.

- Registro **EPC(numero 14)**: Es un registro que guarda la dirección de la instrucción en la que se produjo la interrupción. Se usa junto con la instrucción especial *eret* para retornar a esta dirección al terminar de atender la interrupción.

El teclado y la pantalla: Para activar las interrupciones provocadas por el teclado y la pantalla se deben acceder a sus direcciones mapeadas en memoria y modificar ciertos bits que activan o desactivan la interrupción.

Para activar las interrupciones provocadas por teclado se debe acceder a la memoria **(0xFFFF0000)** y colocar en 1 su bit 1. Para desactivarlo se coloca en 0 el mismo bit. Lo mismo pasa con la pantalla, pero en este caso se debe acceder a la dirección **(0xFFFF0008)**, se repite lo mismo con su bit 1. Ahora bien, ¿Qué pasaría si habilitamos interrupciones en los dispositivos, pero no en el procesador?, En este caso el teclado y la pantalla emitirán las señales de interrupción pero el procesador estará *"sordo"*. Para que preste atención a las interrupciones debemos habilitar su bit 0 en verdadero.

El reloj:

Para activar las interrupciones por reloj debemos activar el bit 15 del registro *status(registro 12)*, y para establecer el temporizador debemos tomar el tiempo actual del procesador a través del registro *count(registro 9)*, sumar el tiempo del temporizador y guardarlo en el registro *compare(registro 11)*.

¿Qué ocurre cuando se produce una interrupción por reloj? Cuando ocurre la interrupción el hardware incrementa el registro Count en cada ciclo hasta alcanzar el valor almacenado en el registro Compare. El hardware detecta la coincidencia y activa el bit 15 del registro Cause para señalar la interrupción del temporizador. Al estar habilitadas las interrupciones en el registro Status (bit 0 y máscara del temporizador activa), el procesador suspende la ejecución normal. El hardware guarda la dirección de la siguiente instrucción en el registro EPC, deshabilita temporalmente las interrupciones globales poniendo en 0 el bit 0 de Status y salta a la dirección 0x80000180. Por otro lado, el software(rutina de servicio) guarda en la pila los registros generales que utilizará para no sobreescribir los datos del programa principal. Se lee el registro Cause para verificar que la interrupción fue causada por el temporizador. Se atiende la interrupción ejecutando las operaciones requeridas y programa un nuevo valor en Compare o modifica Status para que la alarma no se repita. Restaura desde la pila los registros generales guardados al inicio. Ejecuta la instrucción eret, la cual restaura el modo de operación previo y devuelve el control a la dirección almacenada en EPC. Es importante guardar y restaurar en la pila todos los registros generales que la rutina utilice (por ejemplo t0, t1, t2, a0, v0).

Usos del reloj para solucionar poblemas: El reloj lo podemos usar para evitar bucles infinitos, para esto, el temporizador se configura como un mecanismo de supervisión periódica que interrumpe la CPU periódicamente, quitándole el control al programa atrapado para que el sistema operativo o el manejador pueda evaluar el estado y abortar el bucle. Tambien podemos usar el reloj para finalizar programas por tiempo máximo, al iniciar un proceso se establece una alarma con el límite máximo de ejecución, si el tiempo expira antes de que la tarea termine, la interrupción toma el control y da por finalizado el programa por exceso de tiempo. Ahora bien, Lo que debemos hacer si el programa finaliza antes es desarmar o desactivar el temporizador antes de terminar, ya sea apagando el bit correspondiente en el registro Status o reiniciando la configuración de Compare, para evitar que una interrupción pendiente se dispare después de que el programa haya concluido o cuando otro proceso esté en memoria.

Tanto interrupciones como excepciones son interrupciones que alteran el flujo normal de la ejecución de un programa, sin embargo, tienen sus diferencias. Las interrupciones son asíncronas, es decir, no están sincronizadas con el CPU y se producen fuera de este por periféricos de E/S. Si llevamos a cabo el mismo programa estas interrupciones se pueden producir en momentos diferentes.

Ahora bien, las excepciones son interrupciones síncronas provocada por el procesador a la hora de ejecutar una instrucción. Las excepciones se producen porque el procesador detecta alguna anomalía con la instrucción que va a llevando a cabo y detiene el flujo de ejecución para evaluar esta anomalía. Podemos ver la diferencia entre las interrupciones por hardware y las excepciones cuando llevamos a cabo el mismo programa varias veces y notamos que se generan las mismas excepciones en el mismo lugar, esto porque las excepciones están sincronizadas con el procesador.

3.2. Generación de excepciones

Algunas de las principales excepciones se generan por operaciones aritméticas y otras excepciones se producen por acceso a datos o instrucciones invalidad. Ejemplo de excepciones:

- División por cero. El procesador detecta esta operación invalida y la detiene.
- Desbordamiento de memoria. El procesador detecta que el resultado de una operación desborda la capacidad del registro que lo puede almacenar.
- Raíz cuadrada de un número negativo.
- Instrucciones inválidas. El procesador detecta que un programa intenta ejecutar una instrucción para la cual no tiene permiso.
- Violación de acceso. Un programa intenta acceder a una dirección de memoria que le pertenece al sistema operativo o a otro programa.

Ahora bien, de las cinco etapas del pipeline(*IF, ID, EX, MEM, WB*) de MIPS32, cuatro pueden generar excepciones y la única que no lo hace es la etapa *Write Back(WB).*

Instruction Fetch(IF): es la encargada de cargar la instrucción y puede generar diferentes excepciones como cuando una dirección no pertenece a la lógica de MIPS32, es decir, no es una instrucción de 32 bits cuyos dos bits menos significativos siempre son 0.

Instruction Decode(ID): Se encarga de descomponer la instrucción para detectar el tipo de instrucción y los registros a emplear. Puede producirse excepciones en esta etapa cuando el procesador detecta que una instrucción es invalida o el programa no tiene los permisos para hacer eso. Además, instrucciones como *System Call(syscall)* generan excepciones ya que hace una petición directamente al procesador.

Execute: En esta etapa la ALU realiza las operaciones que indica la instrucción, dichas operaciones pueden generar excepciones ante desbordamiento de capacidad y operaciones inválidas como las mencionadas anteriormente.

Memory Access: En esta etapa se accede a la memoria para leer o almacenar datos, lo que puede resultar en excepciones del tipo *address error* cuando se intenta acceder a una direcció de memoria invalida.

3.3. Tratamiento de excepciones

En MIPS32 existen dos maneras de tratar una excepción. Por vector único o vectorizadas por hardware.

En primer lugar, el tratamiento por vector único o despacho por software se da debido a que todas las excepciones e interrupciones transfieren el control a una misma dirección fija de memoria (por defecto 0x80000180). Una rutina general inspecciona mediante software el registro Cause para identificar el motivo exacto y saltar al manejador correspondiente. De esta manera, en esa dirección fija de memoria debemos crear algoritmos que en primer lugar guarden el contexto(los registros que serán empleados y cuyos datos deben protegerse) y luego determinar el tipo de excepción o interrupción que se produjo(como la pulsación de una tecla, un cronometro, etc) y tratarla. Para esto se usan los registros de coprocesador *status y cause*. En el primer ejercicio del presente laboratorio se trato con

la interrupción de teclado, el objetivo era verificar si el usuario presionaba una tecla en mayúscula e imprimirla. Para esto definimos en la dirección 0x80000180 un algoritmo que verificaba si se producía una interrupción por teclado, si esto sucedía verificaba si era una letra mayúscula y en caso de serlo la guardaba en un buffer y retornaba a la instrucción interrumpida gracias al registro EPC y la instrucción especial *eret*.

La otra forma de manejar excepciones es mediante interrupciones vectorizadas por hardware. El tratamiento de excepciones vectorizadas por hardware permite que la CPU salte directamente a una dirección de memoria específica para cada fuente de interrupción, eliminando la necesidad de que una rutina general lea el registro Cause por software para averiguar qué dispositivo solicitó atención. En pocas palabras es un método más complejo de tratar las excepciones y se basa en que a cada interrupción le corresponde una dirección en memoria en donde se puede tratar dicha excepción.

4. Análisis y discusión de resultados

Ambos ejercicios pusieron en practica los conceptos de interrupción y excepciones. El primer ejercicio requería detectar las interrupciones por teclados de letras mayúsculas. Por supuesto, para ello se debió habilitar las interrupciones activando el bit 0 del registro *status*, también activando el bit 1 de la dirección 0xFFFF0000 correspondiente a la memoria mapeada del teclado. Gracias a esto se activaron las interrupciones pertinentes y con cada pulsación este periférico emitía señales que interrumpían el flujo de ejecución del procesador y lo redirigía hacia la rutina de servicio 0x80000180 en donde se guardaban en la pila todos los registros usado allí. Dentro de la rutina de servicio, se evaluaba la procedencia de la interrupción a través del registro *cause(bits 6:2)*, si era cero significaba que procedía del teclado(ya que era la única interrupción activada), luego se extraía el carácter pulsado desde el registro mapeado en memoria del teclado(dirección 0xFFFF0004), al ser caracteres ASCII se evaluaba si pertenecían al rango [65, 90], este rango corresponde a las letras mayúsculas del alfabeto('A' - 'Z'), y se guardaban en el buffer circular y posteriormente vaciado en la consola.

El segundo ejercicio puso en práctica más conceptos, en primer lugar requería dos interrupciones: por teclado y por reloj. Lo que pedía este ejercicio era un sistema de semáforo que se activaba al pulsar la tecla *s* y que luego se activaban temporizadores para cambiar a los diferentes colores del semáforo. Para esta práctica se hizo lo mismo en la lógica de interrupciones por parte del teclado que en la practica anterior, pero también se activo el bit 15 del registro *status* para activar el temporizador, además de establecer los temporizadores con el tiempo pertinentes leyendo el tiempo del procesador a través del registro *count* y cargando el tiempo del temporizador en el registro *compare*. A pesar de ser la misma lógica que el ejercicio anterior requirió un poco más de lineas en la rutina de servicio, por el hecho de prestar mucho cuidado al reloj, es decir, cuando se activa el semáforo en verde no requería de temporizador, por lo cual debía de ser desactivado en el registro *status* para no generar problemas por mantenerse activada.

El presente laboratorio puso en práctica conceptos muy importantes, desde las interrupciones, las excepciones hasta el tratamiento de estas y el cuidado y protección de datos.

5. Bibliografía

- Universidad Complutense de Madrid. Tema 9, interrupciones. https://www.fdi.ucm.es/profesor/jjruz/web2/tems/Curso05_06/EC9.pdf
- LinkedIn. ¿Cómo elegir entre entrada/salida por sondeo y entrada/salida controlada por interrupciones en software embebido?. https://www.linkedin.com/advice/1/how-do-you-choose-between-polling-interrupt-driven?lang=es
- Universidad de Sevilla. (2017, octubre 07). *Gestión de Entrada/Salida*. Sistemás Operativos, Universidad de Sevilla. https://1984.lsi.us.es/wiki-ssoo/index.php/Gesti%C3%B3n_de_Entrada/Salida
- Universidad de Las Palmás de Gran Canaria. (2020). *Guía de Excepciones e Interrupciones en MIPS32 - Lab 1 - 20-21*. https://www.studocu.com/es/document/universidad-de-las-palms-de-gran-canaria/perifericos-e-interfaces/20-21-pi-lab1-30862455?sid=cb1e2b02-50a2-42f7-8894-9b62150f57dd1791567759
- Universidad Tecnológica Nacional Facultad Regional Buenos Aires. (2024). *TP13 U8: Interrupciones y Excepciones en Arquitectura de Computadoras*. https://www.studocu.com/es-ar/document/universidad-tecnologica-nacional-facultad-regional-bu arquitectura-de-computadores/tp13-u8-interrupciones-y-excepciones-en-arquitectur 128749411?sid=cb1e2b02-50a2-42f7-8894-9b62150f57dd1791571188
- Manejo de excepciones. (s.f.). En Wikipedia. Recuperado el 3 de octubre de 2026, de https://es.wikipedia.org/wiki/Manejo_de_excepciones
- Universidad de Palermo. (2026). *Interrupciones y Excepciones en Sistemas Operativos - 0217 APU*. https://www.studocu.com/es-ar/document/universidad-de-palermo/arquitectura-de-computadores/interrupciones-y-excepciones-en-sistemas-operativos 166474119?sid=cb1e2b02-50a2-42f7-8894-9b62150f57dd1791568094